

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

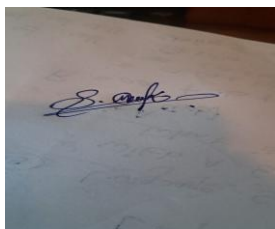
Duration: 30 minutes

Marks: 25

Code of Conduct:

I Shaik Mubarak(192425194)that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

A photograph of a handwritten signature in blue ink on a white piece of paper. The signature is cursive and appears to read 'S. Mubarak'.

Signature of the student:

Name of the student: Shaik Mubarak

QUESTIONS:05

Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Solution:

Given

Loss Function:

$$J(\theta) = \frac{1}{2}(\theta - 5)^2$$

Gradient Descent Update Rule:

$$\theta_{t+1} = \theta_t - \eta \frac{dJ}{d\theta}$$

Since

$$J(\theta) = \frac{1}{2}(\theta - 5)^2$$

Differentiate,

$$\frac{dJ}{d\theta} = \theta - 5$$

Assume

Initial parameter,

$$\theta_0 = 20$$

Case 1: High Learning Rate

Take

$$\eta = 1.2$$

Iteration 1

Gradient

$$20 - 5 = 15$$

Update

$$\begin{aligned}\theta_1 &= 20 - 1.2(15) \\ &= 20 - 18 \\ &= 2\end{aligned}$$

Iteration 2

Gradient

$$2 - 5 = -3$$

Update

$$\begin{aligned}\theta_2 &= 2 - 1.2(-3) \\ &= 2 + 3.6 \\ &= 5.6\end{aligned}$$

Iteration 3

Gradient

$$5.6 - 0.6 = 5.0$$

Update

$$\begin{aligned}\theta_3 &= 5.6 - 1.2(0.6) \\ &= 5.6 - 0.72 \\ &= 4.88\end{aligned}$$

Notice

$$20 \rightarrow 2 \rightarrow 5.6 \rightarrow 4.88$$

The solution oscillates around the minimum.

Apply Learning Rate Decay

Reduce learning rate

$$\eta = 0.2$$

Next iteration

Gradient

$$4.88 - 5 = -0.12$$

Update

$$\begin{aligned}\theta_4 &= 4.88 - 0.2(-0.12) \\ &= 4.904\end{aligned}$$

Now the solution approaches

$$\theta = 5$$

smoothly.

Analysis

High learning rate

✓ Fast movement towards minimum.

After decay

✓ Smaller updates.

✓ Stable convergence.

Risks

If

$$\eta = 3$$

then

$$20 \rightarrow -25 \rightarrow 65 \rightarrow -115$$

The algorithm diverges.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Solution:

Given

Dataset:

$$X = \{x_1, x_2, x_3, \dots, x_n\}$$

Assume

$$X_i \sim N(\mu, \sigma^2)$$

where

- Mean (μ) = Unknown
- Variance (σ^2) = Known

Step 1: Write the Likelihood Function

For a Gaussian distribution,

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

Step 2: Take Log Likelihood

$$\ln L(\mu) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Step 3: Differentiate w.r.t. μ

$$\frac{\partial \ln L}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu)$$

Set equal to zero,

$$\frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0$$

Multiply both sides by σ^2

$$\sum_{i=1}^n (x_i - \mu) = 0$$

Expand,

$$x_1 + x_2 + \dots + x_n - n\mu = 0$$

Therefore,

$$n\mu = \sum_{i=1}^n x_i$$

Hence,

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

This is the Maximum Likelihood Estimator.

Case 1: Dataset Contains Outliers

Suppose

Dataset A

$$\{10, 12, 11, 13, 14\}$$

Estimated mean,

$$\hat{\mu} = \frac{10 + 12 + 11 + 13 + 14}{5} = 12$$

Now include one outlier

Dataset B

$$\{10, 12, 11, 13, 100\}$$

Estimated mean,

$$\hat{\mu} = \frac{146}{5} = 29.2$$

Observation:

True values are around **12**, but the estimate becomes **29.2**.

Therefore,

MLE changes significantly because of one extreme observation.

Case 2: Sample Size Increase

Suppose

Sample 1

$$n = 5$$

Variance

$$\text{Var}(\hat{\mu}) = \frac{\sigma^2}{5}$$

Now

$$n = 100$$

Variance

$$Var(\hat{\mu}) = \frac{\sigma^2}{100}$$

Since

$$\frac{\sigma^2}{100} < \frac{\sigma^2}{5}$$

the estimate becomes much more accurate.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Solution:

Given

Number of features

$$p = 10000$$

Training samples

$$n = 500$$

Since

$$p \gg n$$

High-dimensional learning problem.

Problem Analysis

Initially,

Feature-to-sample ratio

$$\frac{10000}{500} = 20$$

Each sample has 20 times more features than observations.

This leads to

- Overfitting
- High computation
- Poor generalization

Step 1: Feature Selection

Suppose

Initial Features

$$10000$$

After LASSO

$$2500$$

Features removed

$$10000 - 2500 = 7500$$

Percentage removed

$$\frac{7500}{10000} \times 100 = 75\%$$

Step 2: PCA

Apply PCA

Remaining Features

2500

Principal Components

200

Reduction

$$2500 - 200 = 2300$$

Variance retained

95%

Step 3: Regularization

Objective function

$$J(w) = Loss + \lambda ||w||^2$$

Suppose

Loss

25

Regularization

4

Then

$$J(w) = 25 + 4 = 29$$

Large weights receive penalties.

Model complexity decreases.

Step 4: Cross Validation

500 samples

5-Fold Validation

Each fold

Training

400

Testing

100

Average accuracy

Suppose

92%

More reliable than one train-test split.

Inference

Feature Selection



Dimensionality Reduction



Regularization



Cross Validation



Classifier

This pipeline minimizes overfitting and improves generalization.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Solution:

Given

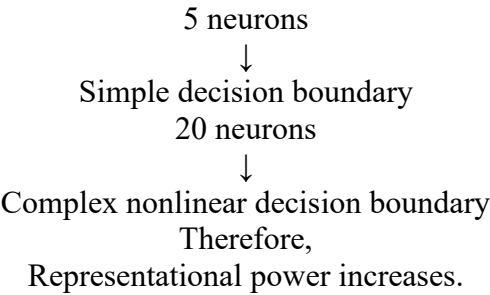
Input Features	10
Output Neurons	2
Case 1	
Hidden Neurons	5
Total weights	
Input to Hidden	$10 \times 5 = 50$
Hidden to Output	$5 \times 2 = 10$
Total	60

Case 2

Increase Hidden Neurons	20
Weights	
Input-Hidden	$10 \times 20 = 200$
Hidden-Output	$20 \times 2 = 40$
Total	240
Increase	$240 - 60 = 180$

More parameters mean higher learning capacity.

Representation Analysis



Overfitting Analysis

Training Accuracy	99%
Testing Accuracy	82%
Difference	$99 - 82 = 17\%$

Large difference

↓

Overfitting.

Generalization

Suppose

Hidden neurons

5

Testing Accuracy

90%

Hidden neurons

100

Testing Accuracy

75%

Although capacity increased,
generalization decreased.

Inference

Increasing hidden neurons increases representational power.

However,

Too many neurons increase model complexity and cause overfitting.

Therefore,

Optimal hidden neurons provide the best balance.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Solution:

Given

Dataset

Features

10000

Training Samples

500

Step 1: Gradient Updates

Gradient update equation

$$w = w - \eta \frac{\partial J}{\partial w}$$

Suppose

Weights

10000

Then

Each iteration updates

10000 gradients.

Higher computation.

Step 2: SGD Convergence

Suppose

Low-dimensional dataset

Iterations

300

High-dimensional dataset

Iterations

2000

Increase

$$2000 - 300 = 1700$$

Training becomes slower.

Step 3: Generalization

Training Accuracy

100%

Testing Accuracy

70%

Difference

30%

Large difference

↓

Model memorizes training data.

↓

Poor generalization.

Technique 1: PCA

Original Features

10000

After PCA

300

Reduction

9700

Computational cost decreases.

Gradient computation becomes faster.

Technique 2: Regularization

Objective Function

$$J(w) = Loss + \lambda ||w||^2$$

Large weights are penalized.

Overfitting decreases.

Generalization improves.

Technique 3: Dropout

Suppose

Hidden Neurons

100

Dropout Rate

50%

During training

Only

50

neurons remain active.

This prevents neuron dependency and improves generalization.